



虚拟内存 - 基础概念

2400012960 祁优扬

前言

- 为什么我们需要虚拟内存？

虚拟内存 (Virtual Memory, **VM**) 为每个进程提供了一个大的、一致的和私有的地址空间。

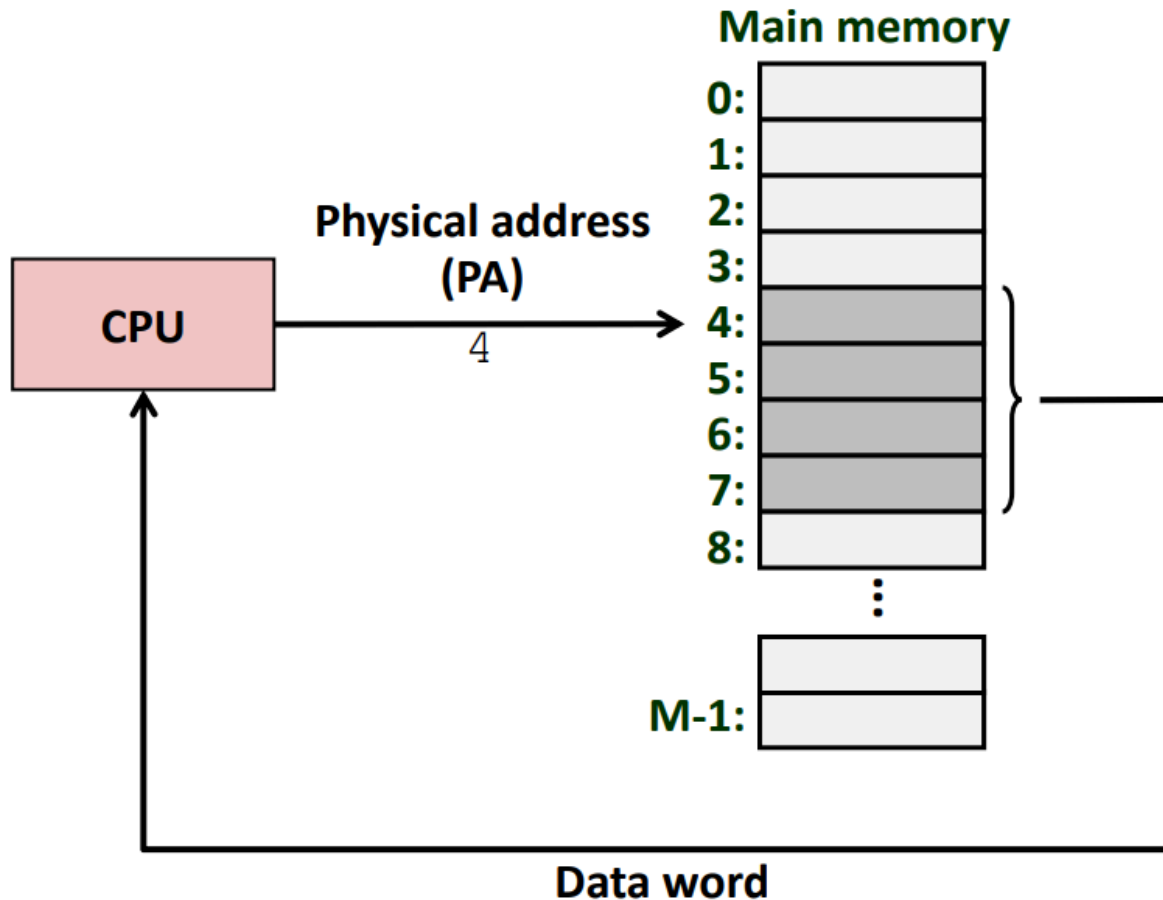
- 大：32 位地址可以表示 $2^{32} = 4\text{G}$ 个字节，64 位地址可以表示 $2^{64} = 16384\text{P}$ 个字节。按照当前主流的 32GB 的内存条算，需要插 $2^{29} = 512\text{M}$ 根内存条才能满足 64 位地址的最大寻址空间，这显然是无法承受的。
- 一致：无论物理内存的实际布局如何（可能非常碎片化），每个进程都认为自己拥有一个连续的线性地址空间。
- 私有：一个进程无法读写另一个进程的私有内存数据，从而防止了进程间的相互干扰，实现了地址空间的隔离和保护。

物理寻址和虚拟寻址

物理寻址

主存被组织成一个由 M 个连续的字节单元组成的数组，每个字节都有一个唯一的**物理地址** (Physical Address, **PA**) 。

CPU 访问内存的最自然的方式就是使用物理地址，这种方式被称为**物理寻址**。

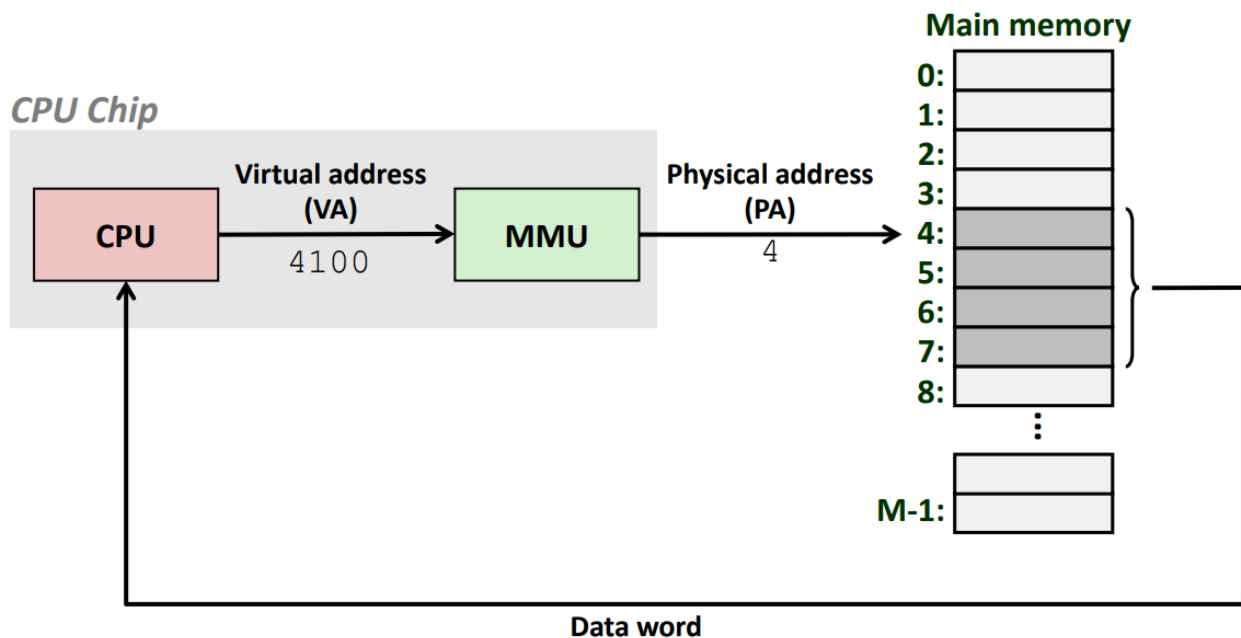


早期的 PC 使用物理寻址，而且数字信号处理器、嵌入式微控制器（在汽车、电梯和数码相框等设备中使用）以及 Cray 超级计算机这样的系统仍在继续使用物理寻址。

虚拟寻址

CPU 通过生成一个**虚拟地址**（Virtual Address, **VA**）来访问主存，在访存时先把这个虚拟地址转换成适当的物理地址，即**地址翻译**。

CPU 上的**内存管理单元**（Memory Management Unit, **MMU**）利用存放在主存中的页表来动态翻译虚拟地址，该表的内容由操作系统管理。



* ARM 体系结构的主要产品线

产品线	虚拟存储	主要应用
“应用”配置：Cortex-A 系列	支持	高性能计算、通用操作系统 ：智能手机、平板电脑、服务器、车
“嵌入式”配置：Cortex-R 系列	不一定	实时、安全关键系统 ：汽车动力系统、制动系统、工业控制、研
“微处理器”配置：Cortex-M 系列	不支持	微控制器、深度嵌入式设备 ：物联网（IoT）、传感器、简单控

地址空间

地址空间是一个非负整数地址的有序集合。我们总是假设使用的是**线性地址空间**，即地址空间中的整数连续。

CPU 从一个有 $N = 2^n$ 个地址的**虚拟地址空间**中生成虚拟地址：

$$\{0, 1, 2, \dots, N - 1\}$$

n 位地址空间即包含 2^n 个地址的虚拟空间。

物理地址空间对应于物理内存的 M 个字节：

$$\{0, 1, 2, \dots, M - 1\}$$

M 不要求是 2 的幂，但为了简化讨论，我们假设 $M = 2^m$ 。

虚拟内存作为缓存的工具

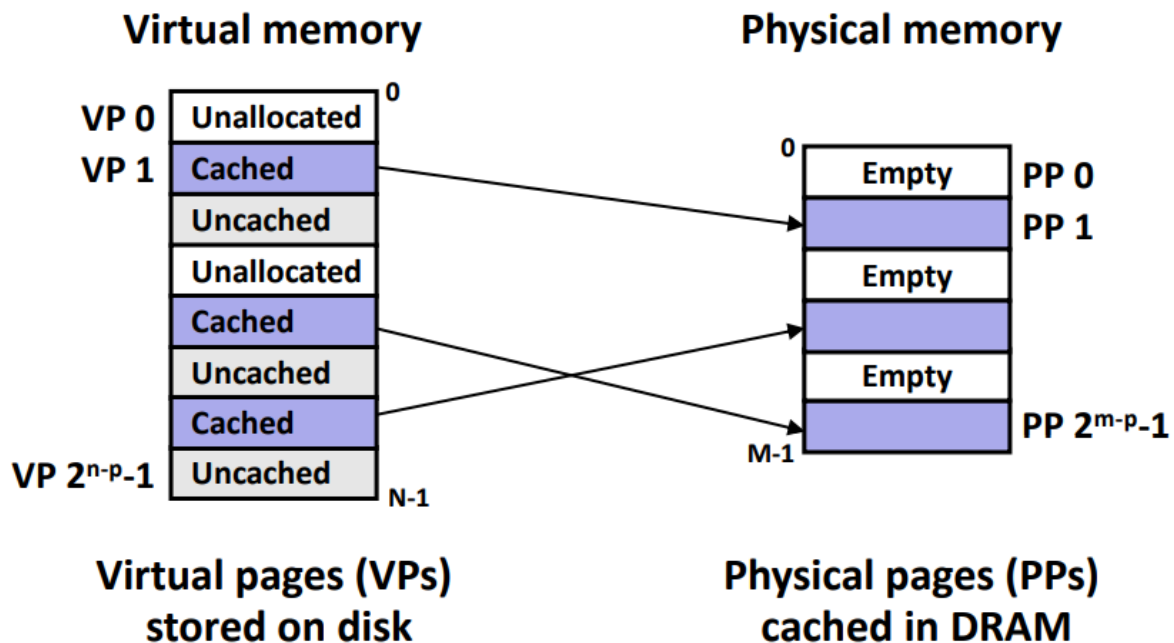
虚拟页和物理页

VM 系统将虚拟内存分割为大小固定的块，称为**虚拟页**（Virtual Page, **VP**），每个虚拟页的大小为 $P = 2^p$ 字节。

类似地，物理内存被分割为**物理页**（Physical Page, **PP**），大小也为 P 字节。

在任意时刻，虚拟页的集合都被分为三个不相交的子集：

- 未分配的（Unallocated）：VM 系统还未分配（创建）的页，不占用任何磁盘空间。
- 缓存的（Cached）：当前已缓存在物理内存中的已分配页。
- 未缓存的（Uncached）：未缓存在物理内存中的已分配页。



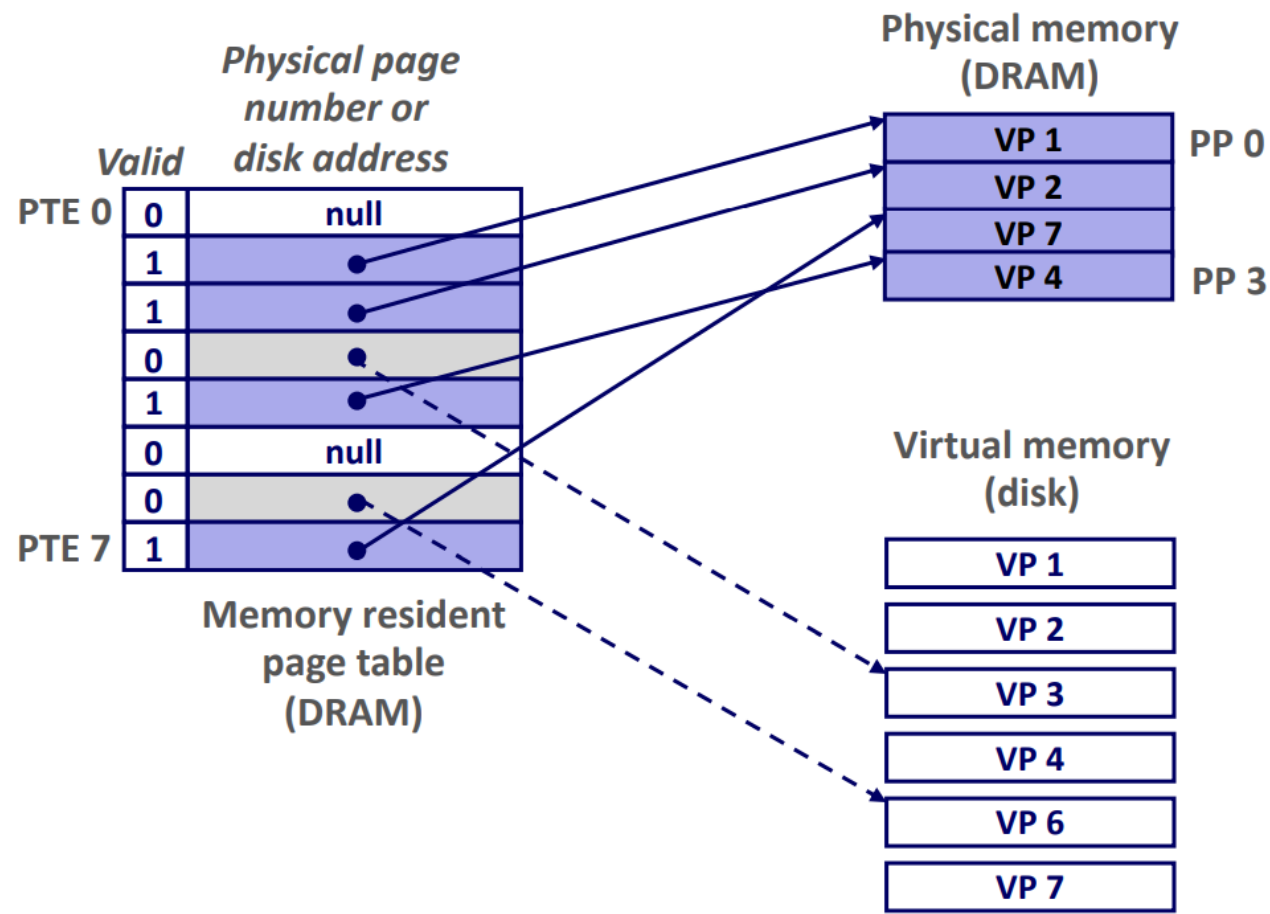
页表

VM 系统必须有某种方法来判定一个虚拟页是否缓存在主存中的某个地方。如果是，VM 系统还必须确定这个虚拟页存放在哪个物理页中。如果否，VM 系统必须确定这个虚拟页存放在磁盘的哪个位置，并在物理内存中选择一个页替换。

操作系统软件、MMU 中的地址翻译硬件和存放在物理内存中的**页表**共同实现这一功能。页表将虚拟页映射到物理页。每次地址翻译硬件将一个虚拟地址转换为物理地址时，都会读取页表。操作系统负责维护页表的内容，以及在磁盘和物理内存之间来回传送页。

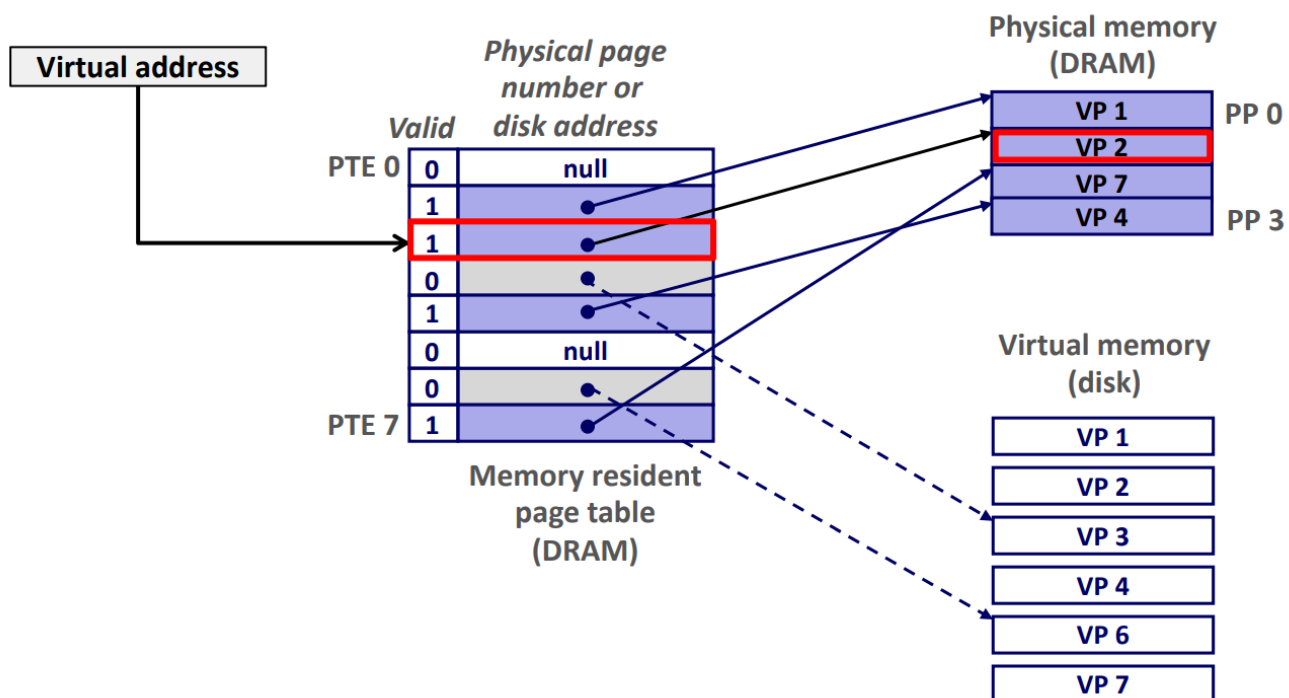
页表是一个**页表条目**（Page Table Entry, **PTE**）的数组。每个 PTE 由一个**有效位**和一个 n 位**地址字段**组成。有效位表明了该虚拟页是否被缓存在物理内存中：

- 如果设置了有效位，那么地址字段就表示物理内存中相应物理页的起始位置。
- 如果没有设置有效位：
 - 如果虚拟页还未分配，那么地址字段为一个空地址。
 - 如果虚拟页已分配，那么地址字段就指向该虚拟页在磁盘上的起始位置。



页命中

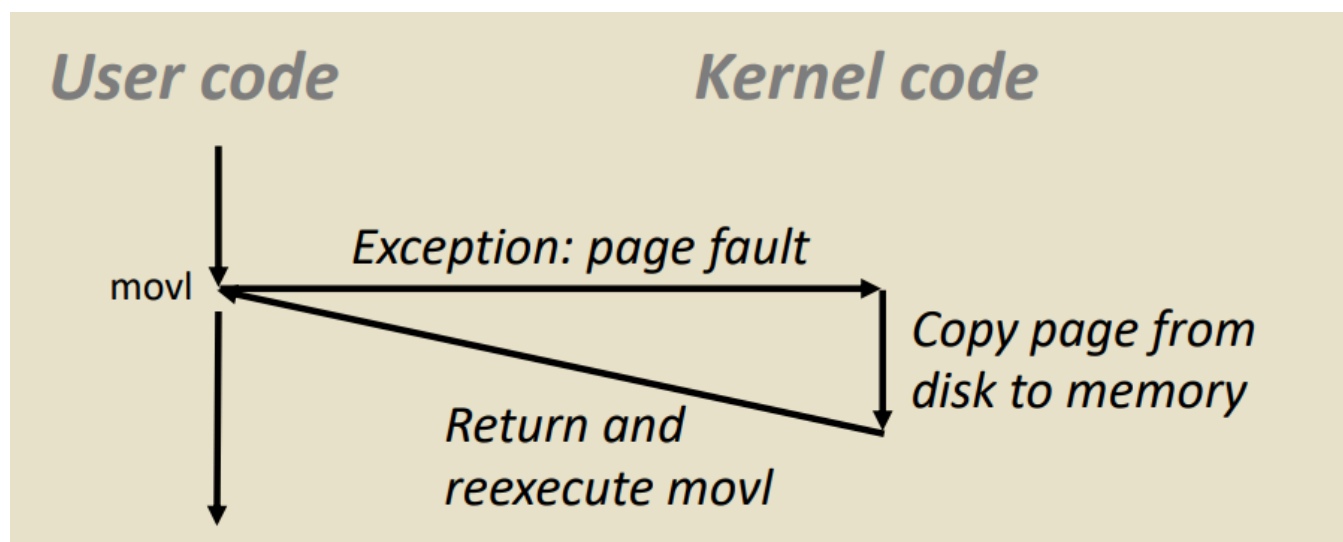
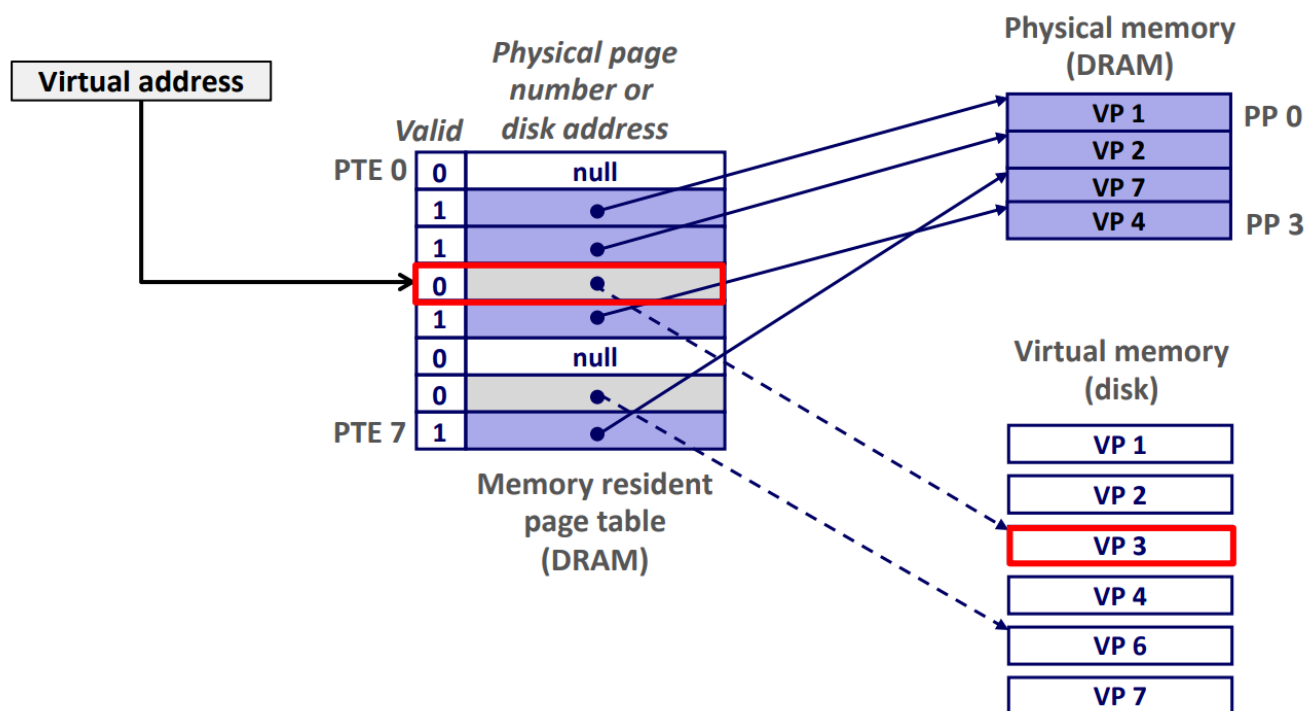
CPU 尝试访问虚拟内存中的一个已缓存在物理内存中的字。它使用 PTE 中的物理内存地址，构造出这个字的物理地址。这种情况被称为**页命中**（Page Hit）。



缺页

CPU 尝试访问虚拟内存中的一个未缓存在物理内存中的字。这种情况被称为**缺页** (Page Fault)。地址翻译硬件检测到缺页，触发一个**缺页异常**。

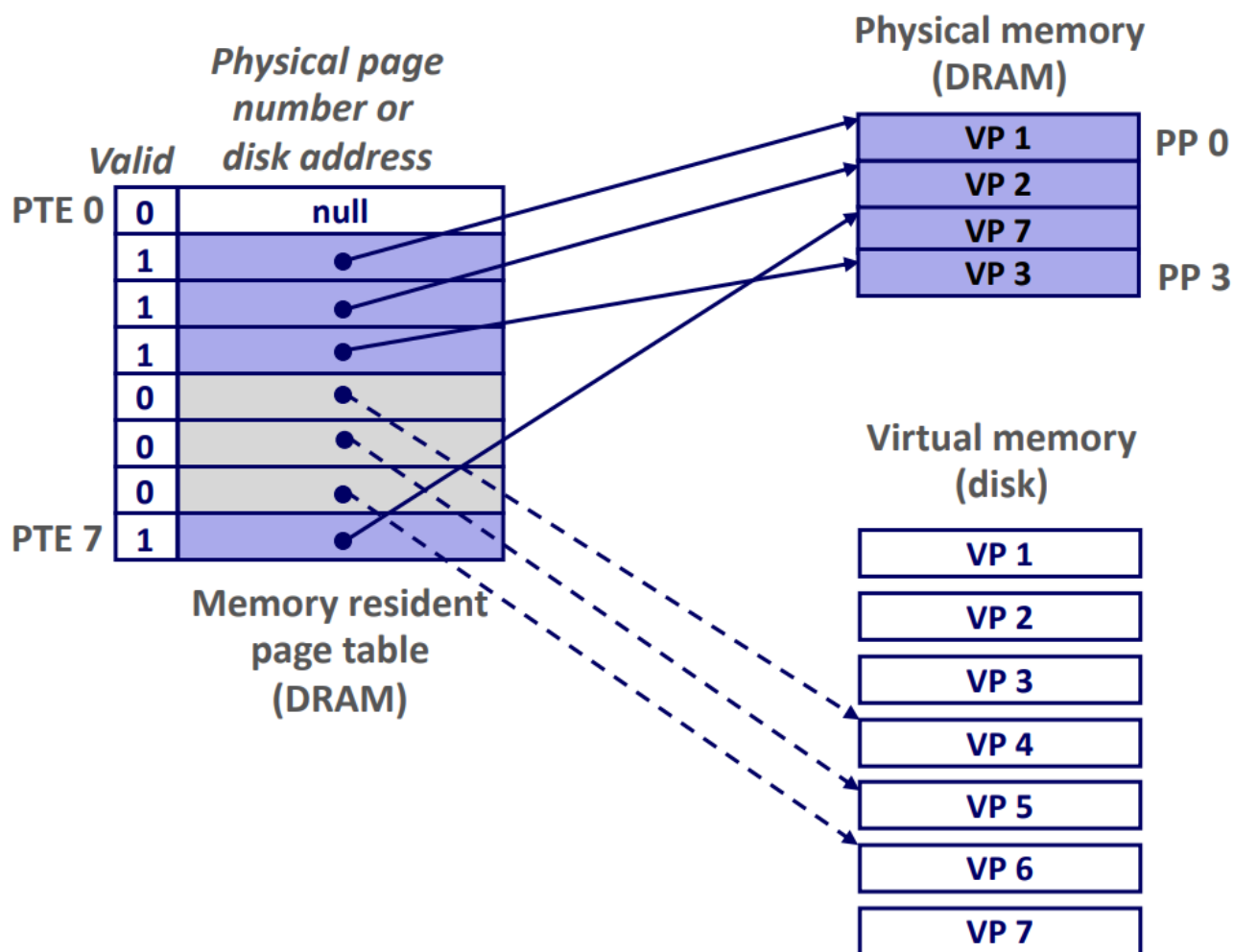
缺页异常调用内核中的缺页异常处理程序，该程序会选择一个物理内存中的牺牲页（如果牺牲页已经被修改了，那么内核会先将它复制回磁盘），修改牺牲页的 PTE，并将要访问的虚拟页从磁盘复制到物理内存，更新要访问的虚拟页的 PTE。当异常处理程序返回时，它会重新启动导致缺页的指令，此时该指令就会得到一个页命中。



一直等待，直到不命中发生，才把页面换入虚拟内存，这种策略被称为**按需页面调度**（Demand Paging）。这种 Lazy 的策略在计算机中被广泛使用。

新分配页

当操作系统分配一个新的虚拟内存页时，在磁盘上创建空间并更新对应的 PTE。随后要访问这个虚拟内存页时，会触发一个缺页异常，从而把它缓存到物理内存。



虚拟内存中的局部性

VM 系统看起来效率很低，但因为**局部性**的存在，虚拟内存实际上工作得相当好。

局部性原则保证，在任意时刻，程序将趋向于在一个较小的**活动页面**集合上工作，这个集合叫做**工作集**或**常驻集合**。在初始开销（冷不命中）之后，对这个工作集的引用将导致命中，不会产生额外的磁盘访问。

只要我们的程序有好的时间局部性，VM 系统就能工作得相当好。但如果工作集的大小超出了物理内存的大小，就会产生**抖动**，此时页面将不断地换进换出，严重影响效率。